ENGINEERING DESIGN DOCUMENT

turing × Kalaam

Standardized Voice-Calling Micro-service Integration (Bolna)

Status	DRAFT — FOR SENIOR REVIEW
Version	0.1
Date	2026-07-09
Author	nishant.sagar@docstribe.com
Reviewers	Senior Engineering
Systems	turing_service · Kalaam backend · kalam_frontend · Bolna (vendor)

Contents

1. Overview & Objectives

`turing_service` ("turing") is being standardized into a **callable micro-service** that any Docstribe platform can invoke to place outbound voice calls through the **Bolna** voice-AI API. **Kalaam** is the first consumer.

A Kalaam operator opens an **AI Voice Calling** page and performs either a **single send** (one patient) or a **campaign** (many patients, selected with Kalaam's existing task-management filter / "bucket" logic — identical to the Saheli/WhatsApp Send flow). They select a Bolna **agent**, fill the agent's required variables, configure **retries**, and press Send. Kalaam's backend calls turing over **hidden, key-authenticated** endpoints; turing places the calls on Bolna. As each call completes, Bolna notifies turing; turing persists the full record (status, transcript, recording, cost) in **its own database** and forwards a **lean outcome** to Kalaam, which stores it in **new tables** in the Kalaam database and links it to the patient.

> **Why:** today turing is a standalone dev tool. This work makes it a production, reusable service with authentication, persistence, and a webhook-based outcome pipeline — and delivers the first end-to-end product surface inside Kalaam.

2. Goals & Non-Goals

GOALS

- turing exposes a stable, versionable, authenticated API for single + batch voice calls.
- turing owns persistence: batches, calls, transcripts, request logs, basic metrics.
- Per-agent variable validation (reject calls missing required variables).
- Event-driven outcome pipeline: Bolna → turing → Kalaam callback.
- Kalaam UI to select patients, agent, variables, retries; single & campaign.
- Kalaam stores a lean per-patient outcome + reference to turing.

NON-GOALS (THIS PHASE)

- LLM transcript analytics / disposition inference in turing.
- Multi-tenant per-client webhook routing.
- Per-client API keys / quota management.
- Inbound (patient-dialed) call handling.
- Campaign analytics dashboards beyond basic metrics.

3. Systems & Terminology

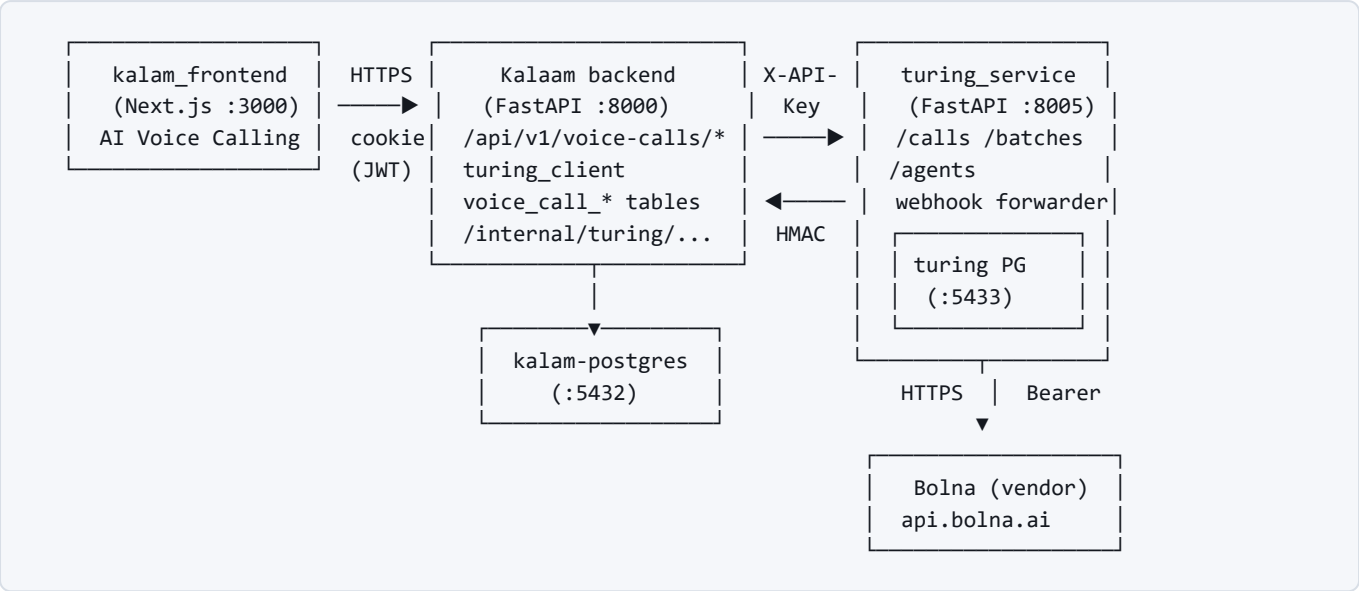
System	Repo / Location	Role
turing_service	`C:\Docstribe\turing_service` (FastAPI, :8005)	Voice-calling micro-service; wraps Bolna; owns call datastore.
Kalaam backend	`C:\Docstribe\kalaam1\kalaam` (FastAPI + Celery, :8000)	Caller; owns patients; stores lean outcomes; receives turing callbacks.
kalam_frontend	`C:\Docstribe\kalam_frontend` (Next.js 16, :3000)	Operator UI; patient selection + campaign wizard.
Bolna	`https://api.bolna.ai` (vendor)	Voice-AI platform that places/records the calls.
demo_call_analyzer	`C:\Docstribe\demo_call_analyzer`	Reusable transcript-analysis logic — future enrichment phase only.

Agent	A Bolna voice agent (prompt + voice + LLM config). Referenced by id.
Execution	One placed call (Bolna `execution_id`).
Batch	A set of calls created from a recipient list (Bolna `batch_id`).
Variable	A `{placeholder}` in the agent prompt filled per call.

4. Confirmed Design Decisions

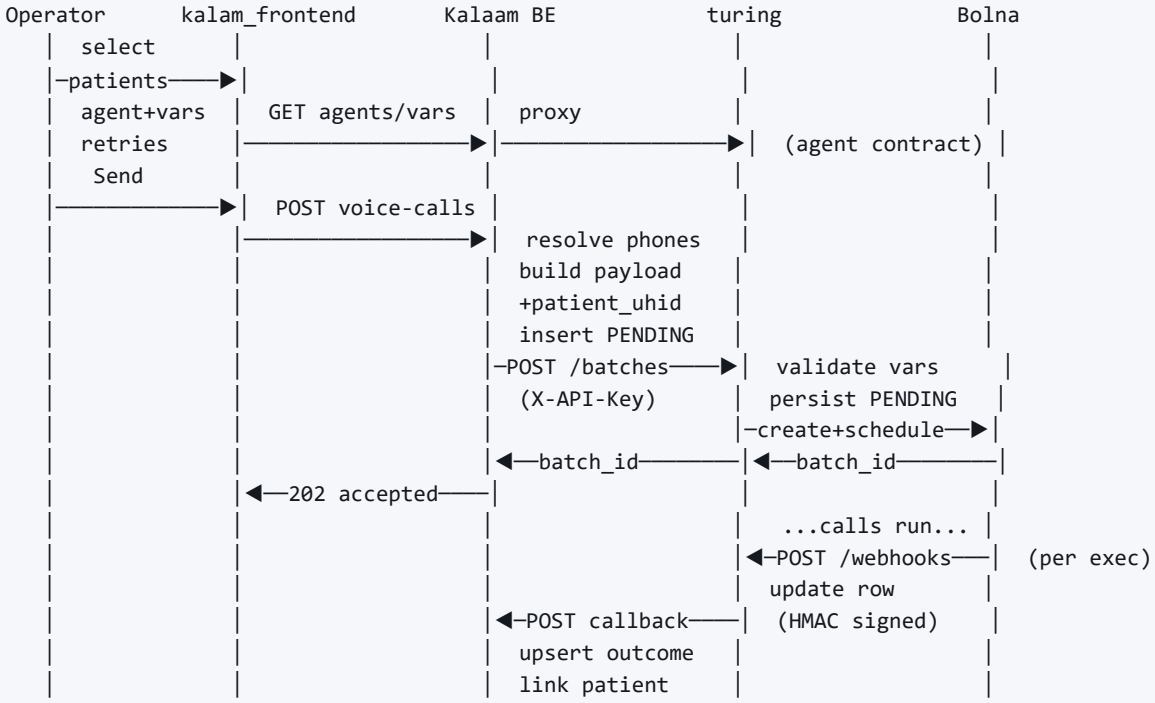
#	Decision	Rationale
D1	**Push model.** Kalaam builds the per-patient payload; turing never reads kalam_db.	Kalaam owns patient data + phone resolution; keeps turing generic and reusable.
D2	**Per-agent variable validation.** turing validates each call against the selected agent's required variables (422 if missing).	Prevents silent broken prompts; agents differ in variables.
D3	**Outcome flow:** Bolna → turing `/webhooks/bolna` → turing persists → turing → Kalaam callback.	Event-driven; encapsulates Bolna specifics inside turing.
D4	**turing is stateful** with a dedicated Postgres (own container). Raw + basic metrics now; LLM analytics later.	turing owns logs/transcripts/analytics as a shared service.
D5	**Kalaam new tables:** lean per-patient outcome + turing reference id.	Avoids duplicating full transcripts; single source of truth in turing.
D6	**Auth:** shared service API key (`X-API-Key`) on turing endpoints; HMAC-signed webhooks.	Simple service-to-service; secret internal URLs.
D7	**Retries:** frontend exposes the full Bolna `retry_config`.	Operator control over retry behavior.

5. High-Level Architecture



The browser never contacts turing directly. Kalaam's frontend proxies to the Kalaam backend (existing BFF pattern); only the Kalaam backend holds turing's base URL and `X-API-Key`. turing is the sole component that talks to Bolna and the only holder of Bolna credentials.

6. End-to-End Sequence



A reconcile poll (turing → Bolna `GET /executions/{id}`; Kalaam → turing) covers missed webhooks and local-dev where turing is not publicly reachable.

7. Standardized Ingestion Contract

Each recipient carries a standard set of fields. `contact_number` is the dialed number (→ Bolna `recipient_phone_number`); the rest are prompt variables. turing validates the subset the *selected agent* actually requires (D2).

Field	Type	Role
contact_number	E.164 string	Dialed number (not a prompt var)
patient_name	string	Prompt variable
doctors_name	string	Prompt variable
department	string	Prompt variable
personalized_symptoms	string	Prompt variable
red_flag_symptoms	string	Prompt variable
last_visit_date	date string	Prompt variable
follow_up_date	date string	Prompt variable
medicine_duration	string	Prompt variable
patient_uhid	string	Linkage ref; round-trips via Bolna `context_details.recipient_data`

> **Review point:** the agent prompts must reference these exact variable names. Kalaam maps patient columns → these names in the wizard (defaults: `phone→contact_number`, `name→patient_name`). Confirm naming with the Bolna agent authors.

8. Component Design — turing_service

8.1 Datastore

Add async SQLAlchemy + Alembic. A dedicated Postgres container (host port **5433**, avoiding kalam-postgres:5432 and docstribe-infra:5434). Tables: `batches`, `calls`, `request_logs` (see §11).

8.2 Authentication

An `X-API-Key` FastAPI dependency validates against `settings.turing_api_keys` (comma-separated). Applied to `/calls`, `/batches`, `/agents`, `/phone-numbers`. `/health` stays open; `/webhooks/bolna` is validated by source-IP allowlist (`13.203.39.153`) and/or a shared secret instead.

8.3 Persistence on create

Existing `calls` / `batches` routers write turing rows (status `pending`) around the Bolna call, recording the Bolna `batch_id` / `execution_id` and the per-recipient rows including `patient_ref`. The existing per-agent variable validation is retained.

8.4 Webhook receiver & forwarder

New `app/routers/webhooks.py` → `POST /webhooks/bolna` updates the matching `calls` row (status, transcript, recording, extracted_data, cost) then `app/services/kalaam_notifier.py` POSTs a normalized *lean* outcome to `KALAAM_WEBHOOK_URL` with an `X-Webhook-Signature` (HMAC-SHA256). On create, turing sets Bolna `webhook_url = TURING_PUBLIC_URL + /webhooks/bolna`.

8.5 Read & metrics endpoints

`GET /batches/{id}`, `/batches/{id}/executions`, `/calls/{id}` served from turing's DB; new `GET /batches/{id}/metrics` (counts, cost, success-rate) and transcript fetch.

8.6 New configuration

`DATABASE_URL`, `TURING_API_KEYS`, `TURING_PUBLIC_URL`, `KALAAM_WEBHOOK_URL`, `KALAAM_WEBHOOK_SECRET`, `BOLNA_WEBHOOK_ALLOWED_IPS`.

9. Component Design — Kalaam Backend

Mirror the existing **Acefone** telephony integration throughout.

New / changed	Purpose	Mirrors
`app/services/turing_client.py`	httpx client to turing (X-API-Key): create_single, create_batch, list_agents, get_agent_variables, get_status.	`services/acefone_client.py`
`app/api/v1/endpoints/voice_calls.py`	POST single, POST batch, GET agents, GET agents/{id}/variables. JWT (`get_current_user`). Resolve phone, build payload, insert pending rows.	`endpoints/calls.py`
2 models + Alembic migration	`voice_call_batches` , `voice_call_records` (see §11).	—
`app/api/internal/turing_webhook.py`	POST `/internal/turing/call-completed` ; verify HMAC; enqueue Celery; 200 fast.	`internal/acefone_webhook.py` + `internal/whatsapp_webhook.py`
`app/workers/voice_call_tasks.py`	Upsert `voice_call_records` by `turing_execution_id` ; link patient by `patient_uhid` else phone; reconcile beat task.	`workers/patient_call_record_tasks.py`
`app/core/config.py` + `.env*`	`TURING_BASE_URL` , `TURING_API_KEY` , `TURING_WEBHOOK_SECRET` .	Acefone config triplet

Phone resolution reuses the existing order `phone → phone_captured → secondary_phone` and normalization helpers in `patient_call_record_service.py` .

10. Component Design — kalam_frontend

Fill the existing (empty) `voice-calls` scaffold, cloning the WhatsApp Send / Campaign flow.

- **Proxy:** `app/api/voice-calls/[...path]/route.ts` (copy of the assignment proxy) → backend `/voice-calls/*` .
- **Service:** `lib/services/voice-calls.service.ts` — `listAgents` , `getAgentVariables` , `startSingle` , `startBatch` .
- **Hook:** `lib/hooks/useVoiceBatch.ts` (TanStack Query, copy of `useAssignment.ts`).
- **UI:** `app/(protected)/voice-calls/page.tsx` + `components/voice-calls/VoiceBatchWizard.tsx` , reusing `useAudienceSelection` , `PatientTable` , `SelectionPanel` .

Wizard steps: Audience (single | campaign via task-mgmt filters) → Agent + Variables (dynamic form from `GET /agents/{id}/variables` ; map each required variable to a patient field or constant) → Retries (full

`retry_config` form) → Review → Send.

11. Data Models

11.1 turing Postgres (source of truth)

```
batches           id (uuid pk) · client · agent_id · from_number · mode(single|batch)
                  · retry_config(jsonb) · bolna_batch_id · status · total_count
                  · valid_count · scheduled_at · created_at · updated_at

calls             id (uuid pk) · batch_id(fk,null) · agent_id · contact_number
                  · patient_ref · bolna_execution_id(unique) · status · transcript
                  · recording_url · extracted_data(jsonb) · cost · duration
                  · hangup_reason · retry_count · raw_payload(jsonb)
                  · created_at · updated_at

request_logs      id (uuid pk) · client · endpoint · method · status_code
                  · latency_ms · created_at
```

11.2 Kalaam new tables (lean outcome + reference)

```
voice_call_batches   id (uuid pk) · created_by_id(fk users) · agent_id · mode
                     · turing_batch_id · retry_config(jsonb) · audience(jsonb)
                     · total_count · status · created_at · updated_at

voice_call_records   id (uuid pk) · batch_id(fk voice_call_batches, null)
                     · patient_id(fk patients, null, indexed) · contact_number
                     · turing_execution_id(unique)  ← idempotent upsert key
                     · status · disposition · recording_url · cost · duration
                     · turing_ref · created_at · updated_at
```

> **Idempotency:** both `calls.bolna_execution_id` and `voice_call_records.turing_execution_id` are unique; webhook handlers upsert on these keys so repeated deliveries are safe.

12. API Contracts

KALAAM → TURING: CREATE BATCH

```
POST /batches            Headers: X-API-Key: <secret>
{
  "agent_id": "<uuid>",
  "from_phone_number": "+9180...",
  "retry_config": { "enabled": true, "max_retries": 2,
                    "retry_on_statuses": ["no-answer","busy"],
                    "retry_on_voicemail": false,
                    "retry_intervals_minutes": [10, 30] },
  "recipients": [
    { "contact_number": "+9199...", "patient_uhid": "UH123",
      "patient_name": "Asha", "doctors_name": "Dr. Rao",
      "department": "Cardiology", "personalized_symptoms": "...",
      "red_flag_symptoms": "...", "last_visit_date": "2026-06-20",
      "follow_up_date": "2026-07-15", "medicine_duration": "30 days" }
  ]
}
→ 201 { "batch_id": "...", "state": "created", "warnings": [] }
→ 422 { "detail": { "error": "missing_required_variables",
                    "missing": ["personalized_symptoms"], ... } }
→ 401 (missing/invalid X-API-Key)
```

TURING → KALAAM: OUTCOME CALLBACK (LEAN)

```
POST /internal/turing/call-completed   Header: X-Webhook-Signature: sha256=<hmac>
{
  "turing_execution_id": "...", "turing_batch_id": "...",
  "patient_uhid": "UH123", "contact_number": "+9199...",
  "status": "completed", "disposition": null,
  "recording_url": "https://...", "cost": 3.2, "duration": 42
}
→ 200 { "received": true }
```

13. Security & Compliance

- **Service auth:** `X-API-Key` on all turing business endpoints; key held only in Kalaam backend env, never in the browser.
- **Webhook integrity:** turing→Kalaam callbacks signed with HMAC-SHA256 over the raw body (mirror Kalaam's WhatsApp webhook verify). Bolna→turing validated by source-IP allowlist.
- **Secrets:** Bolna API key isolated to turing; DB credentials, webhook secrets via env / secret manager. No secrets in the frontend bundle.
- **PII/PHI:** transcripts, recordings, and symptoms are sensitive. turing's DB and Kalaam tables must inherit the same data-handling controls (encryption at rest, access control, retention). Recording URLs are provider-signed; consider expiry.
- **Transport:** TLS between all hops in non-local environments.

14. Reliability, Idempotency & Error Handling

- **Idempotent upserts** on unique execution ids (both sides) — safe webhook retries.
- **Reconcile poll** (Celery beat on Kalaam; turing→Bolna) catches missed webhooks / local-dev where turing is not publicly reachable.

- **Fast-ack webhooks:** receivers return 200 immediately and process asynchronously via Celery.
- **Upstream failures:** turing surfaces Bolna's status code; transport failures → 502. Kalaam persists `pending` before calling turing, so a failed submit is visible and retryable.
- **Partial batch failures:** recorded per-call; batch status derived from execution states.

15. Deployment & Infrastructure

Component	Host port	Notes
turing app	8005	FastAPI/uvicorn.
turing Postgres	5433	New dedicated container (this work).
kalam-postgres	5432	Existing (Kalaam DB, dump loaded).
docstribe-infra Postgres	5434	Untouched.
Kalaam api	8000	Existing.
kalam_frontend	3000	Existing.

> **Webhook reachability:** Bolna cannot reach `localhost`. Live webhooks require turing to be publicly reachable (tunnel/deploy). For local dev, the reconcile poll and a manual "simulate webhook" POST cover the outcome path.

16. Testing & Verification

- **turing (offline, fake Bolna):** Alembic migrations apply to turing PG; create-call/batch writes rows; `X-API-Key` enforced (401 without); missing agent variables → 422.
- **turing webhook:** POST a sample Bolna execution payload to `/webhooks/bolna`; assert row update + signed forward (verify HMAC + lean body).
- **Kalaam BE (live kalam-postgres):** signed sample outcome → `/internal/turing/call-completed`; assert idempotent upsert + patient link by `patient_uhid`. Trigger endpoint with stubbed turing_client → payload build + pending rows; no Bolna call.
- **Frontend:** `tsc --noEmit`; wizard click-through against mocked backend (agent list, variable form, retries, single/campaign payload shapes).
- **End-to-end (gated):** only with explicit approval + 1–2 test numbers; otherwise simulate via the webhook POST.

17. Rollout Plan & Milestones

#	Milestone	Deliverable
M1	turing datastore + auth	PG container, models, Alembic, persist-on-create, `X-API-Key`.
M2	turing outcome pipeline	`/webhooks/bolna`, Kalaam forwarder, read/metrics endpoints.
M3	Kalaam trigger side	New tables, `turing_client`, trigger endpoints, pending rows.
M4	Kalaam callback side	Callback webhook, Celery upsert, reconcile.
M5	Frontend	Proxy, service, hook, voice-calls wizard.
M6	E2E hardening	Gated live test, metrics, docs.

18. Risks & Mitigations

Risk	Impact	Mitigation
Webhook not reachable (NAT/local)	Outcomes never arrive	Reconcile poll fallback; deploy turing publicly for prod.
Agent prompt variable names drift	422s / wrong data	Per-agent validation surfaces mismatches early; wizard mapping step.
PII/PHI in transcripts	Compliance exposure	Encryption, access control, retention; recording URL expiry.
Duplicate webhook deliveries	Double-counting	Idempotent upsert on unique execution ids.
Bolna rate limits / cost overruns	Failed/expensive batches	Respect Bolna concurrency (25); batch guardrails; basic cost metrics.
Two Postgres instances confusion	Ops errors	Distinct ports (5432/5433/5434) documented; named volumes.

19. Open Questions for Reviewers

1. Confirm the Bolna agent prompts use the exact standard variable names in §7.
2. Is `patient_uhid` the right stable linkage key, or should we use Kalaam's internal `patient_id`?
3. Retention policy for transcripts/recordings in turing (PHI) — how long, and who can read?
4. Deployment target + public URL for turing (webhook ingress) and TLS termination.
5. Single shared `X-API-Key` now vs. per-client keys — acceptable for first release?
6. Should campaign scheduling (future-dated batches) be in scope for the first release?

20. Out of Scope / Future Phases

- LLM transcript analytics & disposition inference in turing (reuse `demo_call_analyzer.analyze_transcript`).
- Multi-tenant per-client webhook routing.
- Per-client API keys, quotas, and usage billing.

- Campaign history dashboards & richer analytics.
- Inbound call handling.

End of document — turing × Kalaam Voice Calling Integration Design v0.1 (Draft for review).